

| Course Code  | Course Title                                         | L | R | P | Cr |
|--------------|------------------------------------------------------|---|---|---|----|
|              | AI-Assisted Problem Solving Using Python Programming |   |   |   |    |
| Program Core |                                                      |   |   |   |    |

### Day 7 Session: File Handling & Exception Handling

### Day 7

- File Operations
- Exception Handling

## Topics Covered

- File Operations
- Reading and Writing Files
- File Modes
- Exception Handling
- Try-Except Blocks
- Finally Clause
- Raising Exceptions

## Curiosity Question

### Think Before We Start...

How does WhatsApp save your chats even after you close the app?

How do banking applications recover gracefully from invalid inputs instead of crashing?

### Discussion

- Permanent storage
- Files
- Error handling
- Reliable software

## What is a File?

- A file is a permanent storage medium used to store information.
- **File Handling** in Python is used to **store data permanently in a file and retrieve it later**.

Normally, when a program runs:

- Data is stored in **RAM (temporary memory)**
- When the program stops, data **disappears**

To save data permanently, we use **files**.

## Examples

marks.txt

students.csv

report.pdf

image.jpg

### Python supports

- Text Files (.txt)
- CSV Files (.csv)
- JSON Files (.json)
- Binary Files (.bin)

## Types of Files

### 1. Text Files (.txt)

- Human-readable
- Stores text

#### Examples

student.txt

marks.txt

notes.txt

### 2. Binary Files

- Stores images, videos, audio, executables

#### Examples

image.jpg

video.mp4

audio.mp3

program.exe

### Python file handling:

- Reading files,
- writing files,
- loading data,
- working with and saving data.

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting files.

Python has several functions for creating, reading, updating, and deleting files.

### Python File Open

The key function for working with files in Python is the `open()` function.

The `open()` function takes two parameters; filename, and mode.

Syntax:

```
file = open("filename", "mode")
```

Example:

```
file = open("student.txt", "r")
```

**Closing a File:** `file.close()`

**File Modes:** There are four different methods (modes) for opening a file.

"r" - Read - Default value. Opens a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for writing, creates the file if it does not exist

"x" - Create - Creates the specified file, returns an error if the file exists

| Mode | Description                                          |
|------|------------------------------------------------------|
| r    | Read only                                            |
| w    | Write (creates new file or overwrites existing file) |
| a    | Append data to existing file                         |
| x    | Create a new file (fails if file already exists)     |
| r+   | Read and Write                                       |
| w+   | Write and Read                                       |
| a+   | Append and Read                                      |
| rb   | Read Binary                                          |
| wb   | Write Binary                                         |
| ab   | Append Binary                                        |

The file should be handled as binary or text mode

"t" - Text - Default value. Text mode

"b" - Binary - Binary mode (e.g. images)

## Syntax

To open a file for reading it is enough to specify the name of the file

```
f = open("demofile.txt")
```

The code above is the same as

```
f = open("demofile.txt", "rt")
```

Because "r" for read, and "t" for text are the default values, you do not need to specify them.

**Note: Make sure the file exists, or else you will get an error.**

## Common File Methods

| Method                    | Purpose                           |
|---------------------------|-----------------------------------|
| <code>open()</code>       | Open a file                       |
| <code>close()</code>      | Close a file                      |
| <code>read()</code>       | Read entire file                  |
| <code>readline()</code>   | Read one line                     |
| <code>readlines()</code>  | Read all lines                    |
| <code>write()</code>      | Write text                        |
| <code>writelines()</code> | Write multiple lines              |
| <code>seek()</code>       | Move file pointer                 |
| <code>tell()</code>       | Get current file pointer position |
| <code>flush()</code>      | Write buffered data immediately   |

```
import os
# File name
file_name = "student.txt"
# -----
# 1. Create and Write to a File
# -----
file = open(file_name, "w")      # Creates file if it doesn't exist
file.write("Name : Rahul\n")
file.write("Course : Python\n")
file.write("Marks : 95\n")
file.close()
print("1. File created and data written successfully.")
# -----
# 2. Read the File
# -----
file = open(file_name, "r")
print("\n2. Reading File Content:")
print(file.read())
file.close()
print("File closed after reading.")
# -----
# 3. Append Data
# -----
file = open(file_name, "a")
file.write("College : SR University\n")
file.write("City : Warangal\n")
file.close()
print("\n3. Data appended successfully.")
# -----
# 4. Read Updated File
# -----
file = open(file_name, "r")
print("\n4. Updated File Content:")
print(file.read())
file.close()
```

```
# -----
# 5. Delete the File
# -----
if os.path.exists(file_name):
    os.remove(file_name)
    print("\n5. File deleted successfully.")
else:
    print("\nFile does not exist.")
# -----
# 6. Verify File Deletion
# -----
if not os.path.exists(file_name):
    print("File no longer exists.")
```

Output:|

```
1. File created and data written successfully.
2. Reading File Content:
Name : Rahul
Course : Python
Marks : 95
File closed after reading.
3. Data appended successfully.
4. Updated File Content:
Name : Rahul
Course : Python
Marks : 95
College : SR University
City : Warangal
5. File deleted successfully.
File no longer exists.
```

## What is an Error?

An **error** is a problem in a program that causes it to stop running.

An error is a problem in a program that prevents it from running correctly or producing the expected output.

## Types of Errors in Python

**There are three main types of errors in Python:**

- **Syntax Errors**
- **Logical Errors**
- **Runtime Errors (Exceptions)**

## 1. Syntax Error

A **Syntax Error** occurs when the rules (syntax) of the Python language are violated. These errors are detected **before the program starts executing**.

Example:

```
print("Hello")
```

Output

```
SyntaxError: unexpected EOF while parsing
```

```
Reason: Missing closing parenthesis
```

## Common Causes

- Missing colon (:)
- Missing parentheses ()
- Missing quotes ( ' ' or " ")
- Incorrect indentation
- Misspelled keywords

## 2. Logical Error

A **Logical Error** occurs when the program runs without crashing but produces the **wrong output** due to incorrect logic.

```
length = 10
```

```
breadth = 5
```

```
area = 2 * (length + breadth)
```

```
print(area)
```

**Output**

30

**Expected Output**

50

### 3. Runtime Error (Exception)

A **Runtime Error** occurs while the program is **executing**. The program starts correctly but stops when it encounters an invalid operation.

Example:

```
a = 10  
b = 0  
print(a/b)
```

Output

```
ZeroDivisionError: division by zero
```

The program crashes while executing.

## Common Runtime Errors

| Exception         | Cause                                |
|-------------------|--------------------------------------|
| ZeroDivisionError | Division by zero                     |
| ValueError        | Invalid input value                  |
| TypeError         | Invalid operation between data types |
| IndexError        | Invalid list index                   |
| KeyError          | Dictionary key not found             |
| NameError         | Variable not defined                 |
| FileNotFoundError | File does not exist                  |
| AttributeError    | Invalid object attribute or method   |

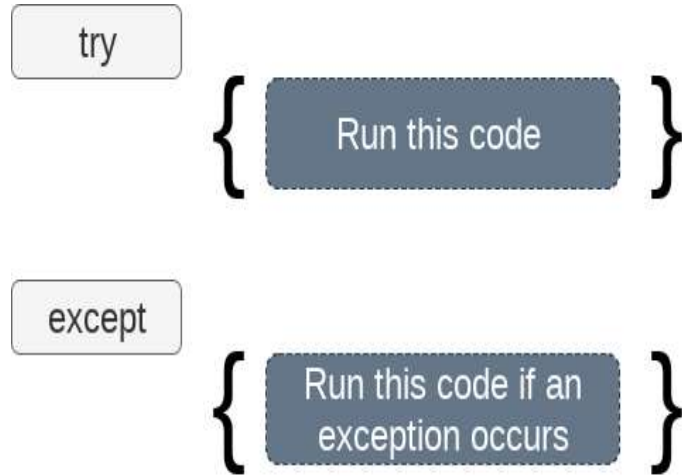
### Exception handling in python:

- When an error occurs, or exception as we call it, Python will normally stop and generate an error message.
- These exceptions can be handled using the **try** statement:

### Python try except

- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.
- The **else** block lets you execute code when there is no error.
- The **finally** block lets you execute code, regardless of the result of the try- and except blocks.

## Python try except:



```

try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2

    numbers = [10, 20, 30]
    index = int(input("Enter index (0-2): "))
    print("Value:", numbers[index])

    print("Result:", result)

except ValueError:
    print("Error: Please enter only integer values.")

except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")

except IndexError:
    print("Error: Invalid list index.")

except Exception as e:
    print("Unexpected Error:", e)

print("Program executed successfully.")
    
```

```

try:
    #block of code

except Exception1:
    #block of code

except Exception2:
    #block of code

#other code
    
```

### Case 1: ValueError

```

Enter first number: 10
Enter second number: abc

Error: Please enter only integer values.
Program executed successfully.
    
```

### Case 2: ZeroDivisionError

```

Enter first number: 10
Enter second number: 0

Error: Division by zero is not allowed.
Program executed successfully.
    
```

### Case 3: IndexError

```

Enter first number: 20
Enter second number: 5
Enter index (0-2): 6

Error: Invalid list index.
Program executed successfully.
    
```

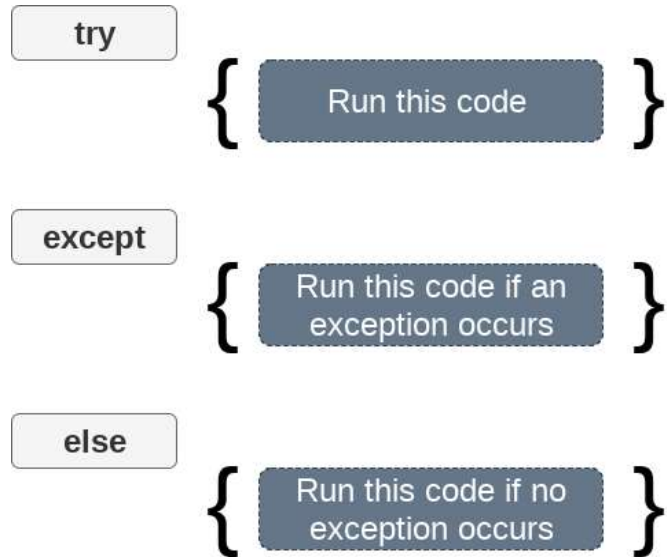
### Case 4: Successful Execution

```

Enter first number: 20
Enter second number: 5
Enter index (0-2): 1

Value: 20
Result: 4.0
Program executed successfully.
    
```

## try except and else block:



try:  
#block of code

except (<Exception 1>,<Exception 2>,<Exception 3>,...<Exception n>):  
#block of code

else:  
#block of code

## Example

```

try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b
    print("a/b = %d"%c)
    # Using exception object with the except statement
except Exception as e:
    print("can't divide by zero")
    print(e)
else:
    print("Hi I am else block")
    
```

## Output:

```

Enter a:0
Enter b:10
a/b = 0
Hi I am else block
    
```

```

Enter a:10
Enter b:0
can't divide by zero
division by zero
    
```

try except else and finally block:

try

Run this code

except

Run this code if an  
exception occurs

else

Run this code if no  
exception occurs

finally

Always run this code

Example:

```
#The finally block gets executed no matter  
#if the try block raises any errors or not:
```

```
try:  
    print(x)  
except:  
    print("Something went wrong")  
finally:  
    print("The 'try except' is finished")
```

```
Something went wrong  
The 'try except' is finished
```



Thank you